UNDEF

FADENA
Facultad de Defensa Nacional

Facultad de Defensa Nacional

Carrera: Lic. en Ciberdefensa

Materia
Sistemas Operativos I

Unidad 1 / Semana 3

Año: 2025

Contenidos

Unidades de medida de información: bit, Byte. Sistemas de numeración: decimal, binario y hexadecimal. Conversión entre sistemas de numeración. Tabla ASCII.

Índice

1. Unidades de medida de información 3
2. Representación digital de datos 5
2.1 Sistemas de numeración 7
2.1.1 Sistema de numeración decimal 7
2.1.2 Sistema de numeración binario 8
2.1.3 Sistema de numeración hexadecimal 9
2.2 Conversión entre sistemas de numeración 10
2.2.1 Conversión de decimal a binario 10
2.2.2 Conversión de binario a decimal 11
2.2.3 Conversión de decimal a hexadecimal 12
2.2.4 Conversión de hexadecimal a decimal 13
2.2.5 Conversión de binario a hexadecimal 13
2.2.6 Conversión de hexadecimal a binario 14
3. Sistemas de Codificación 14
3.1 Evolución de los sistemas de codificación 15
3.2 Representación de datos numéricos 16
3.2.1 Números enteros sin signo 16
3.2.2 Números enteros con signo 16
3.2.2.1 Signo/Magnitud 16
3.2.2.2 Complemento a 1 (ca1) 17
3.2.2.3 Complemento a 2 (ca2) 17
3.2.2.4 Exceso a Z 18
3.2.2.5 Formato BCD (Binary Coded Decimal) 18
3.2.3 Números reales 18
3.2.3.1 Punto decimal fijo 18
3.2.3.2 Punto decimal flotante 18
3.2.3.3 Estándar IEEE-754 32 y 64 bits 19
4. Resumen de la unidad 20

1. Unidades de medida de información

Las unidades de medida de información son aquellas que se utilizan para cuantificar la cantidad de información que se almacena o se transmite en un sistema informático. Al igual que medimos la distancia en metros o el peso en kilogramos, necesitamos unidades para medir la información digital.

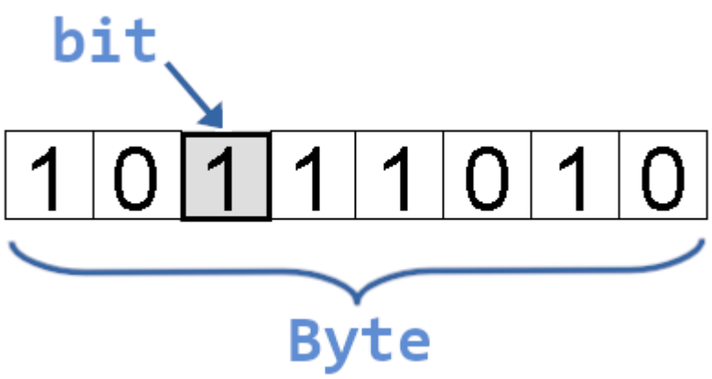
Una computadora no entiende palabras, números, dibujos ni notas musicales, ni incluso letras del alfabeto. De hecho, sólo entienden información que ha sido descompuesta en bits. Un bit, o dígito binario, es la unidad más pequeña de información que una computadora puede procesar.[1]

Unidades básicas:

Bit: acrónimo de Binary Digit (dígito binario), que adquiere el valor 1 o 0 en el sistema numérico binario. En el ámbito de la informática, específicamente en el procesamiento y almacenamiento de datos, el bit es la unidad mínima de información. Imagina un interruptor que solo puede estar encendido (1) o apagado (0); eso es un bit. Físicamente, un bit puede representarse de diversas formas, como un pulso eléctrico que viaja por un circuito o un punto diminuto en un disco magnético.

apagada  encendida 

Byte: en el mundo de las computadoras, la información se almacena y se maneja en unidades llamadas Bytes. Un Byte es como una pequeña caja que puede contener un único carácter, como una letra, un número o un signo de puntuación. Para formar un Byte, se combinan 8 bits, que son las unidades más pequeñas de información, como si fueran 8 interruptores diminutos que pueden estar encendidos (1) o apagados (0).



Dado que un Byte individual solo puede contener una cantidad limitada de información, la capacidad de almacenamiento de las computadoras, como la

[1] Joyanes Aguilar, Luis (2008): Fundamentos de Programación. Algoritmos, estructura de datos y objetos. Cuarta edición, página 17.

memoria RAM o los discos duros, se suele medir en otras unidades que son múltiplos de Bytes.

Para tener en cuenta....

Cuando en las unidades de almacenamiento utilizamos la B (mayúscula) significa Bytes y cuando utilizamos la b (minúscula) hace referencia a bits. Por ejemplo:

MB significa megabytes, pero si se escribe Mb se está indicando megabits.

Múltiplos del Byte

A medida que se manejan cantidades más grandes de datos, se utilizan **múltiplos del Byte** para simplificar las medidas. Estos múltiplos permiten expresar de manera más sencilla las cantidades de datos que se manejan en memoria principal, discos duros o memorias USB, entre otros tipos de memorias secundarias.

En la siguiente tabla podemos observar varias medidas a partir del Byte. Existen más grandes pero con estas podemos trabajar en la mayoría de los casos:

Medida	Símbolo	Equivalente
Byte	B	8 bits
Kilobyte	KB	1024 Bytes
Megabyte	MB	1024 KB
Gigabyte	GB	1024 MB
Terabyte	TB	1024 GB
Petabyte	PB	1024 TB

Exabyte	EB	1024 PB
Zettabyte	ZB	1024 EB

Recordemos que la computadora trabaja en sistema binario, pero es mucho más rápido y sencillo que nosotros como usuarios ingresar los datos en sistema decimal o hexadecimal y que la computadora realice la conversión a binario. Luego nos brinda la salida de información, por supuesto después de su conversión nuevamente a decimal o hexadecimal.

Son ejemplo de uso del sistema hexadecimal, la notación de las direcciones de memoria, codificación de colores para páginas web, entre muchos otros.

2. Representación digital de datos

Las computadoras funcionan con electricidad y por lo tanto los datos y la información que circula en ellas son transmitidos eléctricamente.

El sistema binario es un sistema de numeración en el que los números se representan utilizando solamente las cifras cero y uno (0 y 1). Es el que se utiliza en las computadoras, pues trabajan internamente con dos niveles de voltaje, por lo que su sistema de numeración natural es el sistema binario (encendido 1, apagado 0). **Cuando no circula electricidad equivale un "0"** y **cuando circula electricidad equivale un "1"**. Imaginen que la computadora tiene una lamparita que cuando está encendida indica "1" y cuando está apagada indica "0".

Podemos imaginarnos un bit como una lamparita que puede estar en uno de los siguientes dos estados:

Con 1 bit solamente se pueden representar el 0 y el 1. Pero con 2 bits ya podemos representar 4 números debido a que disponemos de 4 combinaciones diferentes de ceros y unos. Vea la siguiente tabla:

Con 1 bit	Con 2 bits	Con 3 bits
0	00	000
1	01	001
	10	010
	11	100
		011
		101
		110
		111
2^1	2^2	2^3

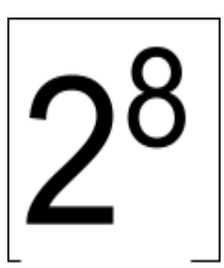
Por ejemplo, se requieren cuatro bits para representar los números decimales del 0 al 15. Es decir que dispondremos de 16 combinaciones de unos y ceros (2^4)

Decimal	Binario
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

Con 6 bits podemos obtener 64 combinaciones, es decir 2^6, con valores de 0 al 63.

¿Por qué utilizamos el Byte?

Para poder representar todos los caracteres que utilizamos en las computadoras (letras en mayúscula y minúscula, números, signos y símbolos), **estos unos y ceros se agrupan de a ocho, conformando un Byte**. Significa entonces que con 8 bits se pueden representar 2^8 (256) diferentes elementos, por ejemplo, los números decimales (0 al 9), las letras mayúsculas del alfabeto (A - Z), las letras minúsculas del alfabeto (a - z), caracteres gráficos, caracteres especiales y caracteres de control.



1 Byte contiene 8 bits y si por cada bit tenemos 2 valores posibles, entonces,

la cantidad de valores posibles del Byte : 2 x 2 x 2 x 2 x 2 x 2 x 2 x 2 = 2^8 = 256

Aquí tenemos un ejemplo:

S	I	S	T	E	M	A	S		O	P	E	R	A	T	I	V	O	S

Ocupa 19 Bytes, que equivalen a 152 bits, es decir 152 unos y ceros combinados.

Observación: Notar que el espacio en blanco ocupa un byte, es decir tiene un valor asignado que se debe representar al momento de ser usado.



Actividad S3-A1

Realice la actividad identificada como *S3-A1* en la sección de actividades del campus.

2.1 Sistemas de numeración

Un sistema de numeración es el conjunto de símbolos y reglas que se utilizan para la representación de datos numéricos o cantidades. Un sistema de numeración se caracteriza por su **base**, que es el número de símbolos distintos que utiliza y además es el coeficiente que determina cuál es el valor de cada símbolo dependiendo de la posición que ocupe.

Como en todo sistema de numeración, el valor de un dígito depende de su posición relativa en el número. Por ejemplo, en el sistema decimal de base diez, el número 3 vale tres, treinta, trescientos o tres mil dependiendo de su posición en el número. Por ejemplo:

3542= 3000 + 500 + 40 + 2

2.1.1 Sistema de numeración decimal

El sistema decimal utiliza un conjunto de símbolos, cuyo significado depende de su **posición relativa** al punto decimal, que en caso de ausencia se supone colocado implícitamente a la derecha.

El hombre ha utilizado el sistema numérico decimal, basado en diez símbolos (0, 1, 2, 3, 4, 5, 6, 7, 8, 9), que, al combinarlos, permiten representar las cantidades. Es por esto que se dice que utiliza la base 10.

El valor de cada dígito está asociado al de una potencia de la base 10, número que coincide con la cantidad de símbolos o dígitos del sistema decimal, y un

exponente igual a la posición que ocupa el dígito menos uno, contando desde la derecha.

Por ejemplo:

$3 * 10^3 + 5*10^2 + 4*10^1 + 2*10^0$

3000 + 500 + 40 + 2 = 3542

Recordemos que todo número (distinto de cero) elevado a la potencia cero da siempre 1. ($X^0 = 1$)

Valor posición	1000 100 10 1
Base^{Exponente}	$10^3 = 1000$ $10^2 = 100$ $10^1 = 10$ $10^0 = 1$
Cantidad de símbolos	10
Símbolos	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Razonamiento	Número típico de dedos igual a diez

Podemos generalizar esto al decir que cualquier número puede ser expresado en forma de polinomio de potencias de su base, lo que se conoce como su *forma polinomial*.

2.1.2 Sistema de numeración binario

Este sistema de base 2 es el más sencillo de todos por poseer solamente dos dígitos, es el sistema que internamente utilizan los circuitos digitales que configuran el hardware de las computadoras.

En una cifra binaria, cada dígito tiene distinto valor dependiendo de la posición que ocupe. El valor de cada posición es el de una potencia de su base (2), elevada a un exponente igual a la posición del dígito menos uno. Se puede observar que, tal y como ocurría con el sistema decimal, la base de la potencia coincide con la cantidad de dígitos utilizados (2) para representar los números.

Por ejemplo, el número binario **1101** se convierte a decimal de la siguiente forma:

$\mathbf{1}*2^3 + \mathbf{1}*2^2 + \mathbf{0}*2^1 + \mathbf{1}*2^0$

8 + 4 + 0 + 1 = 13

y para expresar que ambas cifras describen la misma cantidad lo escribimos así:

$1101_2 = 13_{10}$

Valor posición	128 64 32 16 8 4 2 1
Base Exponente	$2^7 = 128$ $2^3 = 8$ $2^6 = 64$ $2^2 = 4$ $2^5 = 32$ $2^1 = 2$ $2^4 = 16$ $2^0 = 1$
Cantidad de símbolos	2
Símbolos	0, 1
Razonamiento	Los sistemas de voltaje de dos estados (valor binario diferenciado) creados con transistores pueden ser variados, potentes, económicos, pequeños y relativamente inmunes al ruido.

2.1.3 Sistema de numeración hexadecimal

El sistema hexadecimal emplea la base 16. Así, tiene 16 posibles símbolos digitales. Utiliza los dígitos del 0 al 9, más las letras A, B, C, D, E y F como sus 16 símbolos digitales. Es importante recordar que los dígitos hex (abreviatura de hexadecimal) de A a F son equivalentes a los valores decimales de 10 a 15.

Este sistema de numeración es muy utilizado en informática porque simplifica la expresión binaria de los objetos. La siguiente tabla muestra las equivalencias entre el sistema decimal, binario y hexadecimal:

Equivalencias		
Sistema decimal	Sistema Binario	Sistema Hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D

14	1110	E
15	1111	F

Como se puede verificar en la tabla, cada dígito hexadecimal representa un grupo de cuatro dígitos binarios, esto es porque 16 es potencia de 2, dado que 2^4 es 16.

2.2 Conversión entre sistemas de numeración

La conversión de números es esencial para la informática y electrónica, ya que permite que las computadoras, diseñadas para procesar información binaria, se comuniquen con los humanos de manera más comprensible.

Hemos visto que podemos representar números en diferentes sistemas de numeración, usando diferentes bases. Ahora vamos a explorar cómo se pueden convertir números expresados en una base a otra, es decir la **conversión entre sistemas de numeración.**

Los sistemas de numeración más utilizados en informática son: el **decimal (base 10), binario (base 2) y hexadecimal (base 16)**.

Cada sistema de numeración tiene su propia utilidad:

- **Decimal**: El sistema estándar que usamos en la vida cotidiana.
- **Binario**: Fundamental en la informática, ya que los computadores trabajan con 0 y 1.
- **Hexadecimal**: Muy empleado en informática, especialmente en programación y representación de direcciones de memoria.

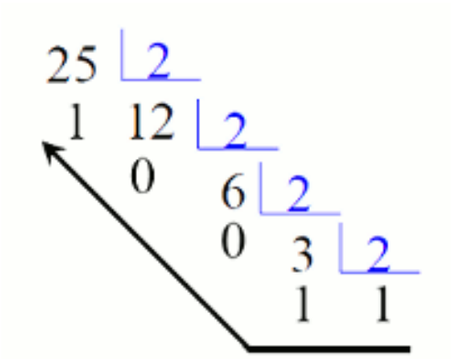
2.2.1 Conversión de decimal a binario

Para realizar esta conversión el método de *división por dos* es muy utilizado y se puede *generalizar para cualquier otra base*.

Debemos realizar los siguientes pasos:

1. Dividir el número decimal por dos
2. Guardar el cociente y el resto.
3. Tomar el cociente anterior y repetir paso 1 hasta que el cociente sea menor que la base.
4. Escribir (concatenar) el último cociente y los restos empezando por el último.

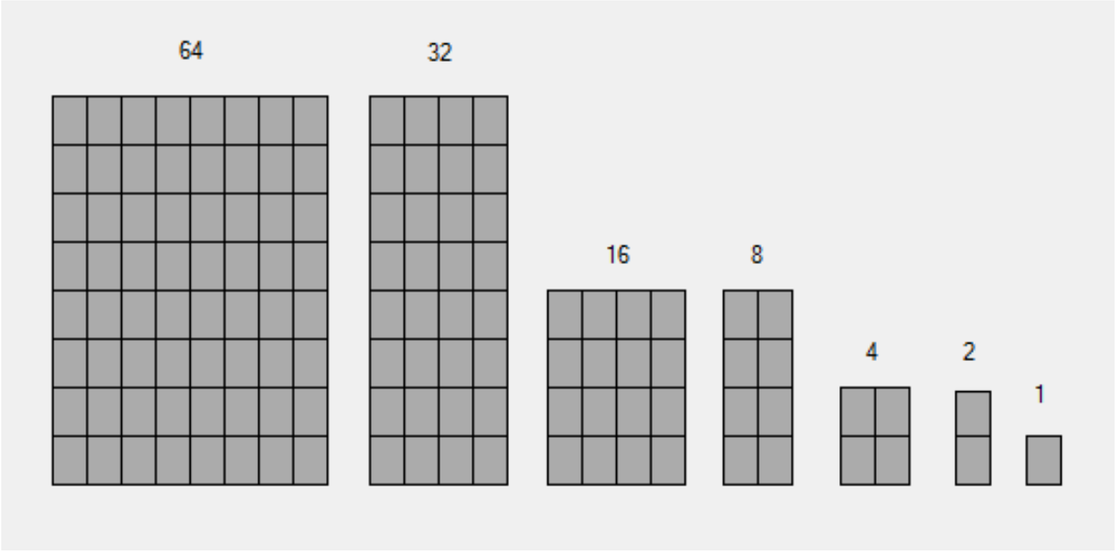
Ejemplo:



Con lo cual, el número 25 en decimal, corresponde a 11001 en binario.

2.2.2 Conversión de binario a decimal

En este caso trabajaremos con la expresión polinómica del número binario a convertir, multiplicando los dígitos por las potencias de 2, desde 2^0 hasta 2^{n-1} en base 10, siendo *n* el número de dígitos. Luego obtener el resultado del polinomio.



Existen infinitos tamaños de agrupamientos, cada uno el doble que el anterior.

El siguiente procedimiento permite realizar la conversión:

- Escribir sobre cada posición binaria su peso decimal (potencia de 2).
- Sumar los pesos de las posiciones cuyos bits valen uno.

Ejemplos de conversión a decimal de los números binarios 10110, 111101 y 10000

16 8 4 2 1		
1 0 1 1 0	= 16 + 4 + 2	= 22 (decimal)
32 16 8 4 2 1		
1 1 1 1 0 1	= 32 + 16 + 8 + 4 + 0 + 1	= 61 (decimal)
32 16 8 4 2 1		
1 0 0 0 0 0	= 32 + 0 + 0 + 0+ 0 + 0	= 32 (decimal)

2.2.3 Conversión de decimal a hexadecimal

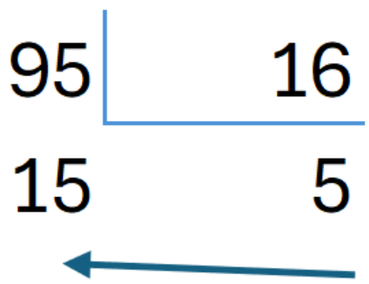
Para realizar esta conversión usaremos el procedimiento ya visto anteriormente para binario, el método de la **división por dieciséis** en este caso.

Debemos realizar los siguientes pasos:

1. Dividir el número decimal por dieciséis
2. Guardar el cociente y el resto.
3. Tomar el cociente anterior y repetir paso 1 hasta que el cociente sea menor que la base.
4. Escribir (concatenar) el último cociente y los restos empezando por el último.

Ejemplo:

Conversión del número 95 decimal a hexadecimal



Tomamos el último cociente y los sucesivos restos, 5 y 15 en ese caso. Sabiendo que el 15 es el dígito F en hexadecimal, el resultado final es **5F**

El número 95 decimal es 5F en hexadecimal.

2.2.4 Conversión de hexadecimal a decimal

En este caso también usaremos la expresión polinómica del número hexadecimal a convertir, multiplicando los dígitos por las potencias de 16, desde 16^0 hasta 16^{n-1} en base 10, siendo *n* el número de dígitos. Luego obtener el resultado del polinomio.

4096 256 16 1		
5 F	= 5x16 + 15 = 95	= 95 (decimal)
4096 256 16 1		
2 1 D	= 2x256 + 1x16 +13	= 541 (decimal)
4096 256 16 1		
1 2 F	= 1x256 + 2x16 + 15	= 303 (decimal)

2.2.5 Conversión de binario a hexadecimal

Para realizar esta conversión, dividimos el número binario en grupos de 4 bits, comenzando por la derecha y luego se convierte cada grupo en un dígito hexadecimal.

El siguiente procedimiento permite realizar la conversión:

- Dividir el número binario en grupos de 4 dígitos, comenzando desde la derecha. Es posible que el último grupo de la izquierda tenga menos de 4 dígitos.
- Convertir cada grupo en un número hexadecimal según la tabla

Ejemplo

16 8 4 2 1 (22 decimal)	1 0 1 1 0	
1 0 1 1 0	= 1 6	= 16 (hexadecimal)
32 16 8 4 2 1 (61 decimal)	1 1 1 1 0 1	
1 1 1 1 0 1	= 3 13	= 3D (hexadecimal)
32 16 8 4 2 1 (32 decimal)	1 0 0 0 0 0	
1 0 0 0 0 0	= 2 0	= 20 (hexadecimal)

2.2.6 Conversión de hexadecimal a binario

Para realizar esta conversión, el procedimiento es convertir cada dígito hexadecimal en binario, según la tabla, si son menos de 4 dígitos se completan con ceros a derecha.

El siguiente procedimiento permite realizar la conversión:

- Tomar cada dígito hexadecimal, comenzando por la derecha
- Convertir dígito hexadecimal en el número binario según la tabla o calculando su valor en decimal y reemplazar por letras cuando el dígito se mayor a 9.

Ejemplo

(63 decimal)	3 F(15)	
3F	= 0011 1111	= 111111 (binario)
(161 decimal)	A(10) 1	
A1	= 1010 0001	= 10100001 (binario)
(211 decimal)	D (13) 3	
D3	= 1101 0011	= 11010011 (binario)



Actividad S3-A2

Realice la actividad identificada como S3-A2 en la sección de actividades del campus.

3. Sistemas de Codificación

Los sistemas de representación de caracteres son fundamentales en la informática y las comunicaciones, ya que permiten la conversión de letras, números y símbolos en un formato que las computadoras pueden procesar y entender. Su evolución ha sido clave en el desarrollo de la informática y las telecomunicaciones.

Desde los primeros días de la computación, se han desarrollado distintos estándares para la representación de caracteres, adaptándose a las necesidades tecnológicas y lingüísticas de cada época. Algunos de los sistemas más importantes incluyen:

- ASCII
- EBCDIC
- ISO-8859
- Unicode
- UTF-8

3.1 Evolución de los sistemas de codificación

En 1963, la ASA (American Standards Association) desarrolló el sistema **ASCII** (American Standard Code for Information Interchange), un estándar de codificación basado en 7 bits, capaz de representar hasta 127 caracteres. En su versión inicial, sólo incluía *letras mayúsculas, números, caracteres especiales y de control*.

En 1967, se amplió el estándar con la incorporación de *letras minúsculas y algunos caracteres adicionales*, dando origen al **US-ASCII**, diseñado exclusivamente para el idioma inglés. En 1969, la ASA pasó a denominarse ANSI (American National Standards Institute).

Mientras tanto, **IBM** había desarrollado entre **1963 y 1964** el sistema **EBCDIC** (Extended Binary Coded Decimal Interchange Code) para sus computadoras mainframe. Sin embargo, a diferencia de ASCII, este sistema no logró una adopción masiva y se utilizó únicamente en el segmento de computadoras de gran porte , con el objetivo de mantener compatibilidad con sistemas anteriores que empleaban el **BCD** (Binary Coded Decimal).

Durante los años ‘80, ASCII fue ampliado a 8 bits, lo que permitió incorporar caracteres especiales de distintos idiomas, dando origen al **ASCII extendido**.

En 1991, se creó el **Unicode Consortium** con el propósito de desarrollar un estándar que pudiera representar todos los idiomas y garantizar la compatibilidad entre sistemas.

El estándar **Unicode** se destaca por su amplitud y flexibilidad, ya que permite codificar caracteres de múltiples idiomas, incluidos los asiáticos, así como símbolos matemáticos. Además, emplea una **codificación variable**, utilizando **8, 16 o 32 bits por carácter**, dependiendo del formato.

Dentro de sus diferentes representaciones, **UTF-8** es el formato más utilizado, especialmente en la web y en aplicaciones modernas, debido a su compatibilidad con ASCII y eficiencia en el almacenamiento de caracteres.

Comparación de formatos Unicode

Formato	Tamaño por carácter	Compatibilidad	Utilización
UTF-8	1 a 4 bytes	Compatible con ASCII	**UTF-8** es el más usado en la web porque es compacto y compatible con ASCII
UTF-16	2 o 4 bytes	No compatible con ASCII	**UTF-16** es eficiente para idiomas como chino y japonés, pero menos común en la web.
UTF-32	4 bytes	No compatible con ASCII	**UTF-32** usa más espacio, pero permite acceso directo a cualquier carácter Unicode

3.2 Representación de datos numéricos

Los sistemas informáticos procesan números utilizando diferentes métodos de codificación. Si bien los caracteres pueden representarse con sistemas como ASCII o Unicode, estos no permiten realizar operaciones matemáticas directamente en la Unidad Aritmético-Lógica (ALU). Por ello, es necesario una codificación específica para los números, y existen varias formas según el tipo de dato numérico.

3.2.1 Números enteros sin signo

Los números enteros sin signo representan únicamente valores positivos, comenzando desde 0 hasta un máximo determinado por el número de bits disponibles. En este caso, cada bit contribuye a la magnitud del número. El esquema es el ya visto en sistemas de numeración binario.

Ejemplo con 4 bits:

- 0000 = 0
- 0001 = 1
- 0010 = 2
- 1111 = 15

El valor máximo se obtiene con la fórmula:

2^n - 1, donde n es la cantidad de bits.

con 1 Byte : 0 a 255

con 2 Bytes : 0 a 65.535

3.2.2 Números enteros con signo

Los números enteros con signo incluyen tanto valores positivos como negativos. Para representarlos internamente se emplean diferentes métodos.

3.2.2.1 Signo/Magnitud

Este esquema usa el bit más significativo (MSB) para indicar el signo:

- 0 → número positivo
- 1 → número negativo

El resto de los bits representan el valor absoluto del número.

Ejemplo con 4 bits:

- 0110 → +6
- 1110 → -6

Este sistema no suele utilizarse porque tiene algunas desventajas:

- Tiene dos representaciones para el cero (**+0**=0000 y **-0**=1000).
- Tanto para la suma y la resta requieren tener en cuenta el signo de los números antes y esto es costoso a nivel de ALU

3.2.2.2 Complemento a 1 (ca1)

Este método invierte todos los bits del número positivo para obtener su versión negativa.

Ejemplo con 4 bits:

- +6 → 0110
- -6 → 1001

Aunque simplifica la inversión de signo, también tiene dos ceros posibles (**+0**=0000 y **-0**=1111), lo que genera ciertas dificultades en cálculos.

3.2.2.3 Complemento a 2 (ca2)

El complemento a 2 es el sistema más utilizado en computación para representar números negativos, ya que permite operar directamente sin realizar ajustes adicionales.

Reglas:

1. Se invierten todos los bits del número positivo.
2. Se suma 1 al resultado.

Ejemplo con 4 bits:

- +6 → 0110
- -6 →1001 +1 →1010

Este método soluciona el problema de tener dos representaciones del cero y simplifica las operaciones matemáticas.

Características

- El intervalo es asimétrico (hay un número negativo más), va desde -2^{n-1} a $+2^{n-1}-1$
- Los n bits representan al nro
- Los positivos comienzan con cero
- Los negativos comienzan con uno
- Hay un solo cero
- Se pueden representar 2^n números distintos (n=número de bits)

con 1 Byte : -128 a +127

con 2 Bytes : -32.768 a +32.767

3.2.2.4 Exceso a Z

En este método, los valores se almacenan sumando un desplazamiento fijo Z. Es común en representaciones de exponentes en IEEE 754.

Ejemplo con exceso a **8** en 4 bits:

- -2 → 0110 (porque -2 + 8 = 6)
- 5 → 1101 (porque 5 + 8 = 13)
- 0 → 1000 (porque 0 + 8 = 8) El cero es el exceso Z

3.2.2.5 Formato BCD (Binary Coded Decimal)

En este formato, cada dígito decimal se guarda en 4 bits, es decir 2 por cada byte.

Ejemplo:

- 53 en BCD → 0101 0011 (cada bloque representa un dígito decimal).

Es útil en aplicaciones financieras donde se requiere exactitud decimal, sin errores de conversión del sistema binario.

3.2.3 Números reales

Los números reales tienen una parte entera y otra parte fraccionaria separadas por el punto o coma decimal. Existen 2 métodos para representarlos en una computadora: coma o punto decimal fijo y flotante.

3.2.3.1 Punto decimal fijo

En los métodos de representación de coma fija se establece un número de bits para la parte entera y número de bits para la parte fraccionaria. Esto limita mucho el rango de números que se pueden representar.

Por ejemplo, se puede decidir que se dediquen 4 bits a la parte entera y 4 para la parte flotante.

En este caso no podría representarse el número 10011 (overflow o desbordamiento) ni tampoco el 0,10011 (underflow o desbordamiento a cero).

3.2.3.2 Punto decimal flotante

Este sistema representa números con una mantisa y un exponente, permitiendo una escala flexible de valores.

Ejemplo de precisión simple (32 bits):

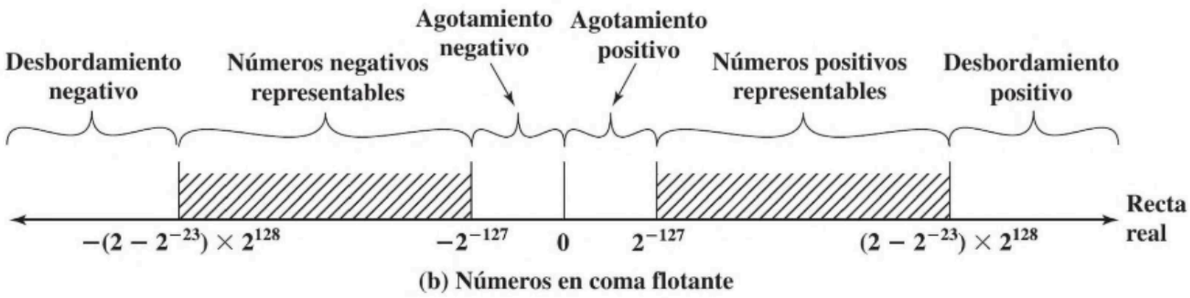
- Signo (1 bit): 0 → positivo
- Exponente (8 bits): 01111111 (exceso a 127)

- Mantisa (23 bits): Los valores fraccionarios del número

Permite representar valores pequeños y grandes sin perder precisión.

Sin embargo, esta representación no cubre toda la recta de números reales porque aparecen 4 zonas que no están cubiertas.

- Números muy grandes (positivos o negativos) generan un desbordamiento (overflow) positivo o negativo según el signo del número.
- Números muy pequeños (positivos o negativos) que generan un desbordamiento a cero (positivo o negativo) y se asume cero



(b) Números en coma flotante

Fuente: Organización y Arquitectura de Computadores, 5ta edición, William Stallings

3.2.3.3 Estándar IEEE-754 32 y 64 bits

La representación en coma flotante más importante es la definida en la norma o estándar 754 del IEEE [IEEE85]. Este estándar se desarrolló para facilitar la portabilidad del software de un procesador a otro y para alentar el desarrollo de programas numéricos sofisticados. Este estándar ha sido ampliamente adoptado y se utiliza prácticamente en todos los procesadores y coprocesadores aritméticos actuales.

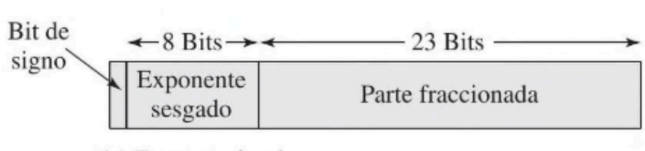
El estándar del IEEE define, tanto el formato simple de 32 bits como el doble de 64 bits, con exponentes de 8 y 11 bits respectivamente. La base implícita siempre es 2.

El exponente está sesgado (exceso) a 127 en 32 bits y a 1023 para 64 bits.

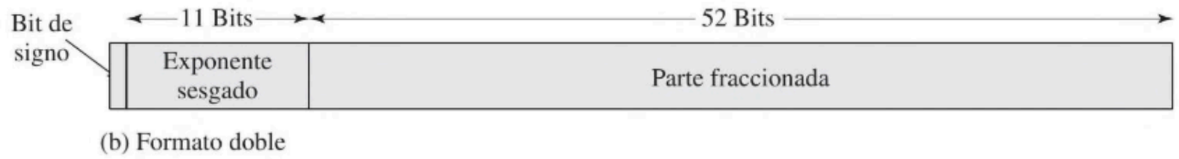
Tiene doble representación del cero, exponente en cero y mantisa en cero pero puede ser positivo o negativo.

Utiliza siempre números normalizados, esto es con el bit de la parte entera de la mantisa o fracción siempre con valor 1, que se omite por conveniencia.

Cuando el exponente está conformado por todos unos y tiene una fracción distinta de cero, se le da el nombre de NaN (Not a Number) para denotar alguna condición de excepción.



(a) Formato simple



(b) Formato doble

Fuente: Organización y Arquitectura de Computadores, 5ta edición, William Stallings

Comparación entre 32 y 64 bits

Característica	IEEE 754 - 32 bits	IEEE 754 - 64 bits
Bits para el exponente	8	11
Sesgo del exponente	127	1023
Bits de mantisa	23	52
Rango máximo	~(10^{38})	~(10^{308})
Precisión decimal	~7 dígitos	~15 dígitos

4. Resumen de la unidad

Las unidades de medida de la información permiten cuantificar la cantidad de datos en los sistemas informáticos, abarcando desde el bit, la unidad mínima de información, hasta escalas más grandes como el petabyte y el exabyte, que aún no tienen un uso masivo en el mercado de tecnología.

Comprender cómo se representa la información dentro de una computadora es esencial. Es importante conocer la existencia y el funcionamiento de los diferentes sistemas de numeración usados en informática y los diversos mecanismos que existen para convertir números entre distintas bases, lo que facilita el trabajo y análisis de datos en diferentes sistemas.

Los estándares internacionales han sido fundamentales en la globalización de los sistemas informáticos y continúan evolucionando, integrando nuevos idiomas con alfabetos muy distintos a los occidentales.

Reconocer la presencia de errores sistemáticos en la representación de datos nos ayuda a interpretar mejor los resultados, especialmente cuando las magnitudes se acercan al límite de las capacidades de representación utilizadas en nuestros sistemas.